NOVA Information Management School

Instituto Superior de Estatística e Gestão de Informação

Universidade Nova de Lisboa

OIL BALANCES WITHOUT HIDDEN ADJUSTMENTS

A Graph-and-Scenario Framework for Crude Oil Logistics Data

Pedro Porfírio

Student Number: 20241283

Dissertation presented as partial requirement for obtaining the Master's degree in Information Management Systems

Supervisor: **Professor Flávio Pinheiro**

May 2026

Oil Balances Without Hidden Adjustments: A Graph-and-Scenario Framework for Crude Oil Logistics Data

Acknowledgements

I am very grateful to my supervisor, Professor Flávio Pinheiro, for his guidance throughout this work, for his readiness to engage with both early ideas and later drafts, and for repeatedly bringing the discussion back to what makes a thesis valuable as a contribution to the literature. Many of the design choices in this framework became clearer through those conversations. I did not merely enjoy working with Professor Pinheiro; I loved it.

I would also like to acknowledge the practitioners working in crude oil trading and analytics whose day-to-day workflows helped define the problem addressed in this thesis. The conventions used by the EIA, the IEA, and the wider public-data ecosystem reflect a great deal of accumulated practical knowledge, and it was important throughout this project to treat that material with the seriousness it deserves.

I would like to dedicate this work to my wife, Ana, who endured many hours waiting for me to finish prototypes or fine-tune the database before we could have dinner, as well as the many movie evenings we missed along the way. For the past 35 years, those have been among the things we most enjoy doing together, which makes her patience and support all the more meaningful to me. Without her by my side, it would have been impossible to accomplish not only this thesis, but so much of what I have done.

I would also like to thank Luiza, Lucas, Mateus, and Julia, my children, who teased me about needing to obtain a proper Master's degree. There you go you four — and I am proud that it is a Master's from NOVA IMS, the best IMS Master's programme in Europe.

Special thanks to Mariana Costa and Manuel Stone for the help and support.

A note of appreciation to NOVA IMS for its patience, encouragement, and support during the writing of this thesis. I would particularly like to thank the administrative staff for their understanding and flexibility in accommodating the personal issues that arose along the way.

Abstract

Crude oil prices are determined in the short run by the physical balance between supply and demand, with inventories acting as the principal short-run price signal. Producers, refiners, traders, and policymakers therefore spend significant resources reconstructing oil balances from public and commercial data published at multiple resolutions by overlapping agencies. The dominant operational technique — Excel workbooks and Python notebooks that ingest data from several sources, reconcile inconsistencies through hard-coded allocation rules, and balance the result with a residual adjustment column — produces usable numbers but buries inconsistency, hides provenance, and cannot be audited or reused without manual rework.

This thesis proposes a representational framework that treats the physical infrastructure of the oil network as orthogonal to the data attached to it. The physical layer — refineries, pipelines, terminals, basins, storage hubs, vessels — is captured once as a persistent graph of assets. Data binds to that graph through per-scenario assignments: each variable on each asset is either observed (bound to a time series) or derived (computed from a formula over other variables), enforced by a schema-level constraint that makes inconsistent state unrepresentable. Reporting residuals are retained as labelled balancing items on the nodes where they arise, not hidden in adjustment columns. Flows that are physically real but unobserved at the per-edge level are declared as latent variables, distinguished from values that are simply missing.

The framework was implemented on the U.S. crude oil network from January 2015 to December 2024 at monthly resolution, with 251 assets, 433 directed flow edges, and 1,870 variables under the active scenario. A head-to-head comparison against the spreadsheet workflow on the same input data shows that the framework produces a materially different output: the persistent six-million-barrel PADD 5 discrepancy lands on a named in-transit corridor node rather than in an unallocated row; the balancing item at PADD 3 spikes visibly during Hurricane Harvey, COVID-19, the Colonial Pipeline shutdown, and the Strategic Petroleum Reserve release — disruptions that the spreadsheet workflow absorbs silently into the adjustment column. The framework's output is consumable by downstream linear programming and graph neural network models without further cleaning.

Keywords: oil balances · crude oil logistics · graph data models · multi-resolution data · mass balance · schema design · linear programming

Resumo

Os preços do petróleo bruto são determinados no curto prazo pelo equilíbrio físico entre a oferta e a procura, com os inventários a actuarem como o principal sinal de preço de curto prazo. Produtores, refinadores, traders e decisores políticos despendem, por isso, recursos significativos na reconstrução de balanços de petróleo a partir de dados públicos e comerciais publicados em múltiplas resoluções por agências sobrepostas. A técnica operacional dominante — folhas de cálculo em Excel e cadernos em Python que importam dados de várias fontes, reconciliam inconsistências através de regras de afectação codificadas manualmente e equilibram o resultado com uma coluna residual de ajuste — produz números utilizáveis, mas oculta inconsistências, dificulta a auditoria e não pode ser reutilizada sem trabalho manual.

Esta tese propõe um quadro representacional que trata a infra-estrutura física da rede petrolífera como ortogonal aos dados que lhe estão associados. A camada física — refinarias, oleodutos, terminais, bacias, hubs de armazenamento, embarcações — é capturada uma única vez como um grafo persistente de activos. Os dados ligam-se a esse grafo através de afectações por cenário: cada variável de cada activo é observada (ligada a uma série temporal) ou derivada (calculada por uma fórmula sobre outras variáveis), garantido por uma restrição ao nível do esquema que torna estados inconsistentes irrepresentáveis. Os resíduos de reporte são preservados como itens de balanço identificados nos nós onde surgem, e não escondidos em colunas de ajuste. Fluxos fisicamente reais, mas não observados ao nível das arestas, são declarados como variáveis latentes, distintas de valores simplesmente em falta.

O quadro foi implementado para a rede de petróleo bruto dos Estados Unidos entre Janeiro de 2015 e Dezembro de 2024 com resolução mensal, contendo 251 activos, 433 arestas direccionadas de fluxo e 1.870 variáveis sob o cenário activo. Uma comparação directa com o fluxo de trabalho em folha de cálculo, sobre os mesmos dados de entrada, mostra que o quadro produz um resultado materialmente diferente: a discrepância persistente de seis milhões de barris no PADD 5 fica num nó nomeado de corredor em trânsito, em vez de numa linha não afectada; o item de balanço no PADD 3 sobe visivelmente durante o Furacão Harvey, a pandemia de COVID-19, o encerramento do Colonial Pipeline e a libertação da Reserva Estratégica de Petróleo — perturbações que o fluxo de trabalho em folha de cálculo absorve silenciosamente na coluna de ajuste. O output do quadro é consumível por modelos de programação linear e de redes neuronais em grafo sem trabalho adicional de limpeza.

Palavras-chave: balanços de petróleo · logística de petróleo bruto · modelos de dados em grafo · dados multi-resolução · balanço de massa · desenho de esquema · programação linear

Disclosure of Use of Generative Artificial Intelligence

The author, Pedro Porfírio, acknowledges the use of generative artificial intelligence tools in the development of this thesis. These tools were used under direct supervision; all generated outputs were critically reviewed, verified for accuracy, and modified as required. The author accepts full responsibility for the validity and originality of the content, and confirms that the use of these tools is consistent with the institutional policies of NOVA Information Management School and with standards of academic integrity.

Tools used: Anthropic Claude (Sonnet and Opus, multiple versions) and OpenAI ChatGPT (GPT-4 and GPT-5 versions).

Activities supported by these tools: (i) initial enumeration of named physical assets in the U.S. crude oil network (refineries, pipelines, terminals) from publicly available sources, reviewed and reconciled by the author against FERC, PHMSA, OGJ refinery surveys, and trade-press sources; (ii) prose drafting and editing support, where the author retained authorship of all argumentative content; (iii) code-writing assistance for the data-ingest pipelines, the resolver implementation, and the SQL view layer, with the author reviewing every commit; (iv) summarisation support for the literature review, with the author independently verifying each citation against the cited source.

Activities not supported by these tools: the design of the framework's principles, the schema, the resolver's dispatch rules, the consistency claims, the case-study comparison, and the interpretation of all numerical results — these are the work of the author.

Contents

List of Figures

List of Tables

Acronyms

AIS	Automatic Identification System (vessel tracking)
API	American Petroleum Institute (crude grade gravity scale)
ARIMA	AutoRegressive Integrated Moving Average
DCRNN	Diffusion Convolutional Recurrent Neural Network
EIA	U.S. Energy Information Administration
FERC	Federal Energy Regulatory Commission
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GNN	Graph Neural Network
IEA	International Energy Agency
kbbl	Thousand barrels
kbd	Thousand barrels per day
LOCF	Last-Observation-Carried-Forward
LOOP	Louisiana Offshore Oil Port
LP	Linear Programming
mbd	Million barrels per day
mmbbl	Million barrels
MILP	Mixed-Integer Linear Programming
NBER	National Bureau of Economic Research
OGJ	Oil & Gas Journal
PADD	Petroleum Administration for Defense District
PHMSA	Pipeline and Hazardous Materials Safety Administration
SPR	Strategic Petroleum Reserve
STEO	Short-Term Energy Outlook (EIA publication)
STGCN	Spatio-Temporal Graph Convolutional Network
TGAT	Temporal Graph Attention Network
TGN	Temporal Graph Network
VAR	Vector AutoRegression
WPSR	Weekly Petroleum Status Report
WTI	West Texas Intermediate (crude grade benchmark)

Chapter 1

Introduction

1.1 Background and motivation

Crude oil is one of the most volatile commodities in global markets, but it is also one of the most closely monitored. Producers, governments, refiners, and traders interact through spot cargoes, term contracts, and standardised futures linked to regional benchmarks. Even so, those financial instruments ultimately depend on the physical balance between supply and demand for crude itself. For that reason, serious quantitative work on oil markets usually starts with the physical system that produces, transports, stores, and consumes the commodity.

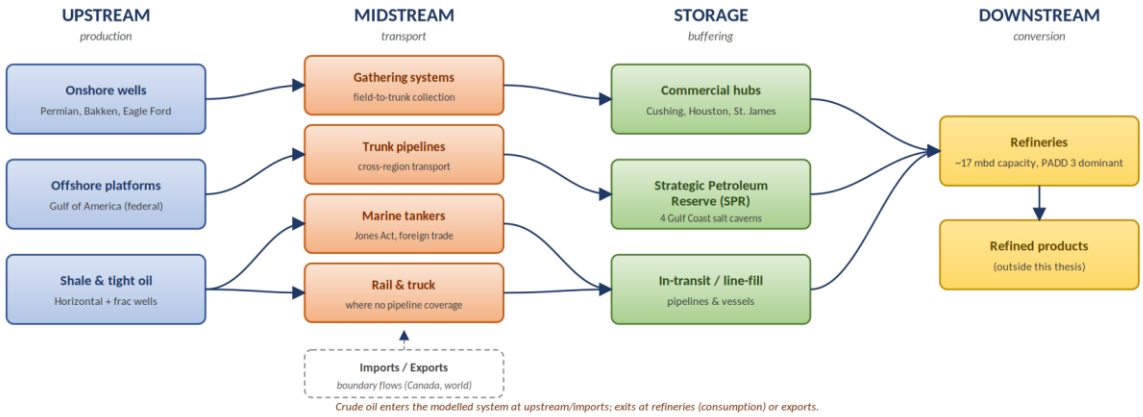

Figure 1.1. *The crude oil physical supply chain: upstream production (onshore wells, offshore platforms, shale and tight oil), through midstream gathering and transport (pipelines, tankers, rail and truck), through storage (terminals and SPR), into downstream refining and refined products.*

In the short run, the physical layer of an oil network is rather constant. On the upstream side, well output is constrained by geology, reservoir conditions, and capital that was committed years before the current period. The economic literature is explicit on the consequence: the short-run price elasticity of oil supply is very low, because there is a lag between investment and production, the costs of adjusting extraction rates are convex, and existing fields show decreasing returns (Bornstein, Krusell, and Rebelo, 2017; Hamilton, 2009). On the downstream side, refineries operate in narrow design windows. They are built for particular throughput ranges and crude slates, and short-term operation is structured around a small set of discrete modes with significant transition cost; this is why operations-research treatments of refineries model mode selection rather than continuous throughput control (Pinto, Joly, and Moro, 2000; Joly, Moro, and Pinto, 2002; Fernandes, Relvas, and Barbosa-Póvoa, 2013). Industry data is consistent with this picture: U.S. refinery utilization rates have hovered between 87% and 95% for most of the past decade, with a practical lower limit — the “turndown rate” — of approximately 65–70% below which units

cannot run economically. Transport adds further delay, with pipelines, ports, and tankers introducing transit times of days to months (Stopford, 2009; Kazemi and Szmerekovsky, 2015). Production and consumption therefore hold close to their operating set-points most of the time, and meaningful changes on either side usually require lead time, capital, and coordination across several parts of the chain (Lima, Relvas, and Barbosa-Póvoa, 2016).

Most short-term mismatches between supply and demand are step changes — shocks to one side that the other side cannot follow. A refinery enters unplanned maintenance, and its crude intake drops abruptly; the upstream production feeding it does not slow correspondingly, because wells and gathering systems cannot be throttled on a few days' notice. A hurricane shuts platforms and refineries on the Gulf Coast at the same time, but the imbalance between the two sides of the balance is rarely symmetric. A pipeline outage interrupts a particular corridor while production at its upstream end and demand at its downstream end carry on as before. In each case, one side of the balance step-changes while the other side stays put, because that is what physical infrastructure does in the short run.

Inventories absorb the resulting gap, and the change in stocks is the observable signature of every such shock. Tank farms, storage hubs, and strategic petroleum reserves are the buffer between what is produced and what is consumed at any given moment, and the level of stocks is therefore one of the principal determinants of short-term crude price dynamics (Working, 1949; Pindyck, 2001; Kilian and Murphy, 2014; U.S. Energy Information Administration, 2024). When stocks are comfortable, shocks can be absorbed with limited price impact. When they become tight, prices rise; when they become excessive, prices soften; and when usable storage is effectively full, the marginal barrel has nowhere to go. The April 2020 WTI front-month settlement at −37 USD per barrel — a price at which the seller pays the buyer to take delivery — was a highly visible demonstration of that last limit. Because the mismatches arrive as shocks rather than as drift, inventory changes are not noise around a trend; they are the recoverable trace of identifiable events, and reading them correctly is what makes a balance informative.

Two ideas from Pindyck (2001) make this connection precise and are particularly load-bearing for the work that follows. The first is that inventories act as the buffer that decouples production from consumption, so spot prices reflect the flow imbalance between the two rather than the level of either; price formation is therefore inseparable from the dynamics of stocks. The second is that production facilities — wells, refineries, and the pipelines that connect them — behave as call options on the underlying commodity, which is why their economic value responds to volatility the way it does. Both ideas map directly onto the framework developed in this thesis: inventories appear as explicit state variables at every physical node, and the structural identity of each producing or consuming asset is preserved as a first-class entity rather than collapsed into aggregate flow.

The operational consequence is direct: if one cannot measure how inventories change, one cannot understand what is happening to price. Stock changes reveal whether a shock has been absorbed by drawing down storage or has spilled into the market, whether refinery throughput is keeping pace with crude availability, and whether a flow that nominally took place has arrived at its destination. Producing oil balances that track stock changes correctly is therefore not a back-office accounting exercise; it is the upstream condition for any quantitative work on oil prices.

That upstream condition is harder to meet than it appears, because the data does not arrive in a shape that matches the physical flow. Public and commercial sources publish at granularities that do not line up with the operational network. The U.S. Energy Information Administration releases monthly series at national, district (PADD), refining-district, state, and named-basin levels. The International Energy Agency publishes country and regional aggregates. Commercial providers — Genscape, Kpler, IIR Energy, S&P Global Commodity Insights — add higher-frequency and finer-granularity views from satellite imagery, AIS feeds, and operator surveys. The same physical quantity often appears at several resolutions at once, and the resolutions do not always reconcile cleanly. Reporting boundaries are administrative rather than operational; some flows are directly measured, and others can only be inferred from aggregate constraints; inventories often close only at coarser levels than the production they are supposed to balance against; and the topology itself changes over time through real events such as pipeline reversals, terminal commissioning, and refinery shutdowns. The structural mismatch between how the system flows and how the data describes it is the central friction this thesis addresses.

The current state of the art for handling that friction in practice is ad hoc. Analysts maintain Excel workbooks or pandas notebooks in which production, consumption, flows, and stocks are loaded from multiple sources, reconciled through hard-coded allocation rules, and balanced with a residual term that absorbs whatever does not add up. Those workflows produce usable numbers, but they bury inconsistency inside the residual and embed each analyst's assumptions in formula cells that cannot be audited from the outside. They cannot be passed directly into an optimisation model or a machine-learning pipeline without further hand-cleaning, and two analysts working from the same data routinely produce different outputs.

The balancing item. The residual term that appears at the end of every oil balance deserves explicit attention from the outset, because it is both unavoidable and informative. The International Energy Agency already publishes a balancing item in its supply–demand tables, defined as the gap between observed stock changes and the sum of observed flows. The convention treats that gap as data: a labelled variable on the published balance, whose size and sign reveal where the reporting system is and is not closing cleanly. When the practitioner spreadsheet workflow hides the same residual in an unlabelled adjustment column, that information is lost — the bottom row of the balance reads zero by construction, but the signal that produced the residual is no longer recoverable. A representation that aims to support

quantitative work on inventories must therefore not only track production, consumption, and flows; it must also retain the balancing item as a first-class variable wherever the data fails to close. Predicting changes in inventories means, equivalently, predicting the production and consumption flows that drive them and the balancing item that accommodates whatever the data cannot resolve.

Modelling tools used to study systems like this fall into three broad groups. Operations research — linear, mixed-integer, and stochastic programming — is mathematically rigorous and well suited to strategic design and tactical planning, but it usually relies on simplified, bespoke representations that leave out much of the operational detail visible in the real network. Classical time-series methods such as ARIMA, VAR, and more recent neural forecasting models are typically applied to individual variables — refinery throughput, regional inventories, benchmark prices — without explicitly modelling the network structure that links those variables together. Over the last decade a third line of work has emerged at their intersection. Graph neural networks and graph representation learning offer a way to model systems whose behaviour depends both on local time dynamics and on a broader relational structure, and recent surveys (Wu et al., 2020; Jin et al., 2024) document how quickly this literature has expanded across traffic networks, power systems, and, more recently, supply chains (Kosasih and Brintrup, 2022; Wasi et al., 2024; Ahn et al., 2024). What makes these methods especially relevant here is that they can bring together two aspects of the problem that classical operations research and standard time-series approaches often handle separately: the network itself and the temporal behaviour of the variables attached to it.

Before any of these modelling tools can be applied to the inventory problem, however, the same upstream question appears: how should the network itself be represented? A representation that can host multi-resolution data without forced reconciliation, that retains the balancing item as a labelled variable rather than absorbing it into an adjustment column, and that admits structural change without requiring a wholesale rebuild, would be useful not only for the optimisation and forecasting work positioned as the framework's downstream consumers, but also for the analysts who currently connect public data to those models through manual workflows.

1.2 Problem statement

Despite the maturity of the three research streams outlined above, there is still no widely adopted way of representing crude oil logistics networks that meets the needs of downstream users. Existing approaches do not deal cleanly with multi-resolution data without relying on ad hoc reconciliation. They do not enforce single attribution at schema level, do not preserve the provenance of derived quantities across the full resolution chain, and do not clearly separate the slowly changing physical topology from the faster-changing data assignments attached to it.

Anyone who has worked with energy data will recognise the practitioner workflow that fills this gap. Analysts often maintain Excel workbooks or pandas notebooks where production, consumption, flows, and stocks for each PADD are loaded from a multitude of sources, reconciled through hard-coded allocation rules, and balanced with a residual adjustment term that absorbs whatever does not add up. Those workflows can produce useful numbers, and they are deeply embedded in day-to-day market analysis, but they do so by burying inconsistency inside the adjustment term. They are also difficult to generalise: each workbook reflects its author's assumptions, and none can be passed directly into an optimisation model or a machine-learning pipeline without further manual cleaning.

The central problem this thesis addresses is therefore: can we design a graph representation of the physical flow of oil that is permanent in structure, accepts different combinations of multi-resolution data, and supports the prediction of changes in inventories — including the balancing item that absorbs whatever the data cannot resolve — in a form consumable by classical optimisation models (linear programmes) and by modern machine-learning models (graph neural networks) without further hand-cleaning?

This thesis answers that question by developing, implementing, and validating an asset-centric temporal graph framework organised around a small set of design principles, together with a deterministic resolver that produces a single per-(scenario, variable, observation date) value table that downstream consumers can read directly. The framework is then demonstrated head-to-head against the dominant current technique on the same input data.

1.3 Research objectives

The research pursues three main objectives:

To design a graph schema that represents the crude oil logistics network at the level of physical assets, captures both structural relationships and time-series dynamics, enforces single attribution and partition closure as schema-level guarantees, and accommodates multi-resolution public data without treating reconciliation residuals as primary data.

To implement that schema as a working system, including a relational database representation, a deterministic resolver that produces the per-(variable, date) data tensor, a layered view structure for downstream consumers, and a complete starter instance covering the U.S. crude oil network from 2015 onwards.

To demonstrate that the framework produces materially more honest balances than the dominant current technique on the same input data, through a head-to-head case study in steady state and under

stress, and to assess its readiness for further downstream consumers such as graph neural networks for forecasting, which are left for future work.

1.4 Scope and delimitations

This research focuses on the representational framework and its first downstream consumers. The asset graph is built at the level of named physical assets wherever public data allows, with residuals used for un-named tail production. The geographic scope is the United States, the temporal resolution is monthly, and the initial implementation treats crude oil as a single commodity grade. The schema is designed to be extendable along all three dimensions — to other grades and refined products, to other countries, and to finer temporal granularity — and those extension paths are discussed where relevant. Even so, the implemented instance in this thesis is the U.S. monthly crude oil graph from January 2015 to December 2024 inclusive.

Two important areas are explicitly outside the scope of the present work and are positioned as future work in Chapter 6. The first is forecasting. Forecasting is treated in this thesis as a downstream use case that the framework is designed to support but does not itself implement. Chapter 6 explains why the framework is ready for forecasting work and outlines what that next step would involve. The second area is production deployment. The implementation developed here is a working research artefact rather than a production system, so issues such as hardware and software selection, user interfaces, monitoring, and access control are left aside.

Two pieces of vocabulary used throughout deserve flagging upfront. The framework distinguishes assets from nodes. An asset is a primitive entity: a refinery, a pipeline, a basin, an export terminal. A node is how that asset appears in a particular graph. Each node binds to exactly one asset, and the same asset can appear in multiple graphs without being duplicated in the underlying data. The starter scenario uses a single graph, so in the current implementation each asset has exactly one node, but the separation is designed in: it lets per-grade, per-operator, or per-period views over the same physical reality be added later without restructuring the asset registry.

Assets carry a kind flag with two values. A physical asset is a real, named entity with capacity, geographic coordinates, configuration, and flow edges to its neighbours; refineries, pipelines, terminals, basins, gathering networks, and storage hubs are all physical assets. An abstract asset is an aggregation view over the physical layer (observational aggregates such as district views, basin views, refining-district views, and stock-decomposition aggregates) and has no flow edges of its own. Abstract assets exist to host roll-up observations and partition relationships and constraints of other nodes; their relationship to the physical layer is declared through formula inputs rather than through edges. Mass balance is enforced at every

physical asset, and abstract assets inherit it as a corollary of aggregation — they do not introduce intrinsic constraints unless explicitly stated.

Two extensions are worth flagging here because they recur in conversations about the framework. The first is per-grade decomposition: by treating commodity as a further dimension on every variable, each tank farm carries inventory per (asset, grade), and the evolving blend in storage is recoverable as the running sum of grade-specific inflows minus grade-specific offtakes — a per-(tank, grade) inventory series rather than a single aggregate. The second is refinery yield: the framework expresses refined-product output as a derived variable through formula and formula inputs, so a refinery's gasoline production can be defined as a fixed fraction of its consumption of a particular grade (gasoline production = 0.35 × crude consumption of grade X), with the yield coefficients themselves potentially scenario-specific. Both extensions sit on top of the schema as constructed; neither requires structural change.

1.5 Thesis structure

The remainder of the thesis is organised as follows. Chapter 2 reviews the techniques currently used to produce oil balances — official statistical balances, the practitioner spreadsheet workflow, optimisation models, and graph and machine-learning approaches — and identifies the gap that the thesis addresses. Chapter 3 gives a focused domain background on the U.S. crude oil network, with particular attention to its multi-resolution reporting structure.

Chapter 4 presents the framework as five design principles, organised around the orthogonality of physical infrastructure and data. Chapter 5 is the central comparative chapter: a head-to-head case study against the spreadsheet workflow on the same input data, in steady state and under stress. Chapter 6 concludes with the contribution, the limitations, and a discussion of future work, including the framework's readiness for graph neural network forecasting as a natural next consumer.

Three annexes provide supporting reference material. Annex A documents the implementation — the schema, the starter U.S. crude oil graph, the resolver, and the consistency audits. Annex B catalogues the variables included in the starter scenario. Annex C offers a primer on graph neural networks for readers who want to follow the GNN-readiness discussion in Chapter 6 in more detail.

Chapter 2

Current techniques for producing oil balances

This chapter reviews the techniques in current use for producing oil balances. Each technique is described on its own terms, with its strengths acknowledged, and its limitations stated. The chapter is organised around four families: official statistical balances published by national and international agencies; the practitioner workflow built on spreadsheets and notebooks; optimisation models from the operations research literature; and graph and machine learning approaches from the recent computer science literature. The chapter closes with the gap that the thesis addresses.

2.1 Official statistical balances

The U.S. Energy Information Administration is the dominant public source for the U.S. crude oil network. Its Monthly Petroleum Supply Reporting System publishes production, refinery runs, stocks, imports, exports, and inter-district movements at several resolutions: national totals, the five Petroleum Administration for Defence Districts (PADDs), refining districts that subdivide each PADD, and named basins and states within each PADD. The Weekly Petroleum Status Report provides a higher-frequency view of a smaller set of series. Across this catalogue, the same physical quantity is often published at more than one resolution simultaneously, with reconciliation between the levels left to the user.

The International Energy Agency follows a similar pattern at the international level. Its monthly Oil Market Report and annual World Energy Balances present supply, demand, refining, stocks, and trade for OECD countries and major non-OECD producers. A point of methodology that matters for what follows: the IEA retains an explicit balancing item in its supply–demand tables, defined as the residual between observed stock changes and the sum of observed flows. The convention treats the residual as information rather than as noise to be hidden. The size and sign of the balancing item indicate where the reporting system is not closing cleanly, and the IEA does not try to make that issue disappear by silently folding it into nearby quantities. This thesis follows the same convention, applied at every physical node in the network rather than only at country-level aggregates.

Official statistical balances have well-known strengths: standardised methodology, free access, international comparability, decades of historical coverage. Their limitations are also well known. The monthly granularity is too coarse for several operational questions; reporting boundaries are fixed by administrative convenience rather than by physical structure; named facilities are exposed only patchily, with most series aggregated to the PADD or refining-district level; and the same publication often contains series at incompatible resolutions, with no canonical guidance on which is authoritative when they

disagree. The published balance is a useful starting point for any further analysis, but it is not by itself the operational answer that traders, refiners, or analysts need.

2.2 The practitioner spreadsheet workflow

The dominant operational technique for producing oil balances inside trading firms, consultancies, integrated oil companies, and government agencies is a workbook or notebook that ingests the relevant published series, reconciles them across resolutions through a set of allocation rules embedded in formulas, and presents a balance to the user. Anyone who has worked with energy data will recognise the pattern. Production at named basins is summed into a PADD total, which is compared against the published PADD aggregate. Imports and exports per PADD are read from the relevant series and combined with refinery throughput and stock changes. Whatever does not add up is absorbed into a residual column, usually labelled “adjustment”, “statistical difference”, “unallocated”, or simply left without a label. The bottom row of the balance reads zero by construction.

These workflows have real strengths. They are fast to build, easy to modify, embedded in the daily work of the people who use them, and produce a usable answer in seconds. They also accumulate institutional knowledge: a workbook that has been refined over years of trading desk operation often encodes subtleties about EIA's revision conventions, custody-transfer timing, or particular pipelines that are not written down anywhere else. The technique is not a methodological mistake; it is the natural response to the underlying data problem, and the framework developed in this thesis is best understood as a more disciplined version of the same instinct.

Their limitations are equally real. The first is that reconciliation rules are implicit and inconsistent across analysts. Each workbook embeds its author's assumptions about how to allocate the PADD-level production residual across named basins, how to handle the difference between gross and net imports, how to attribute the SPR drawdown to commercial stocks. Two analysts working on the same EIA data produce different derived quantities because their reconciliation choices differ, and the differences are not visible in the bottom line.

The second limitation is that the residual adjustment column hides information. The IEA's convention of publishing the balancing item at the national level exists because reporting frictions are systemic — custody-transfer timing, refinery process losses, vessel measurement differences, in-transit allocations — and the size of the residual carries information about where the reporting system is failing to close. A workbook that absorbs the residual into an unlabelled adjustment cell loses that signal. The bottom row reads zero, but the analyst no longer knows whether the period balanced because the data is clean or because the adjustment column did its work.

The third limitation is provenance. A derived quantity in a typical workbook depends on a chain of allocations and adjustments distributed across dozens of cells, and tracing any specific value back to its inputs requires reverse engineering the workbook. This makes the workflow hard to audit, hard to hand over to a new analyst, and hard to extend to new data sources. When EIA introduces a finer-resolution series — for example, when PADD-level stocks were decomposed into commercial and SPR portions — incorporating it into a legacy workbook requires modifying the allocation logic in several places, often introducing inconsistencies with the resolution that was previously authoritative.

The case study in Chapter 5 compares the framework developed in this thesis against this workflow directly, on the same input data, in two specific scenarios. The comparison makes the three limitations concrete: a structural discrepancy that the workflow leaves in an unlabelled adjustment cell lands on a named node in the framework, and a disruption month that the workflow absorbs silently into the adjustment column produces a visible spike in a labelled balancing item in the framework.

2.3 Optimisation models

A second family of techniques approaches the network through mathematical programming. The literature on crude oil and refined-product supply chains is extensive, dominated by mixed-integer linear programming and its stochastic extensions. Fernandes, Relvas, and Barbosa-Póvoa (2013) develop a deterministic MILP for the strategic design of multi-entity, multi-echelon, multi-product downstream petroleum supply chains. Kazemi and Szmerekovsky (2015) extend this line of work to multimodal transportation, showing that the joint treatment of pipeline, rail, marine, and truck transport can materially change the optimal solution compared with single-mode formulations. Ghaithan, Attia, and Duffuaa (2017) move toward tactical planning with a multi-objective formulation in an integrated oil and gas setting. On the uncertainty side, Ribas, Hamacher, and Street (2010) develop stochastic programming models for integrated oil-chain planning under uncertainty. Lima, Relvas, and Barbosa-Póvoa (2016) provide a broad review of the downstream oil supply chain management literature, noting that the level of detail at which optimisation models are built often does not match the level of detail at which real input data is available.

The methodological strength of this literature is that the formulations are mathematically rigorous and supported by mature solver infrastructure. The structural commitment that runs through the family is the level of abstraction at which the network is represented: nodes are typically large aggregates (refineries, demand centres, production regions), and edges are abstract connections defined in terms of cost, capacity, or lead time. That representation is well suited to the strategic and tactical questions these models are designed to answer, such as where to locate capacity or how to allocate supply across regions. It is less helpful for the upstream question this thesis addresses, which is how to represent the network in a way that admits multi-resolution data and supports several different downstream consumers without

modification. Each optimisation paper builds its own bespoke representation; the representation is a means to the model's end, not a research artefact in its own right.

2.4 Graph and machine learning approaches

A third family of techniques applies graph data models and graph representation learning to commodity and logistics networks. The applied literature is recent. Kosasih and Brintrup (2022) show that graph neural networks can predict hidden links in supply chain networks, revealing dependencies that aggregate-level data tends to miss. Wasi, Islam, and others (2024) introduce SupplyGraph, the first benchmark dataset explicitly designed for GNN-based supply chain analytics, with tasks including demand forecasting, production planning, and product classification. Ahn, Yoon, and Park (2024) develop a probabilistic GNN model for supply and inventory prediction using data from a global consumer-goods company, showing that attention-based graph architectures can improve predictive performance by capturing cascading effects across nodes.

Behind these applied studies sits a methodological literature on graph representation learning. Foundational work on random-walk embeddings (Perozzi, Al-Rfou, and Skiena, 2014; Grover and Leskovec, 2016) and on graph neural networks more directly (Kipf and Welling, 2017; Veličković et al., 2018; Hamilton, Ying, and Leskovec, 2017) established the methodological base. Surveys by Wu et al. (2020) and Jin et al. (2024) provide useful overviews of the field. Temporal extensions such as TGAT (Xu et al., 2020), TGN (Rossi et al., 2020), and hybrid architectures like DCRNN (Li et al., 2018) and STGCN (Yu, Yin, and Zhu, 2018) make the temporal dimension an explicit part of the modelling framework. The traffic-forecasting literature in particular has produced architectures that fit logistics networks structurally well, because traffic networks share with crude oil networks a largely stable topology, clearly identified nodes, and forecasting targets that depend both on local time dynamics and on dependencies that propagate through the network.

These studies are important because they demonstrate that graph methods can add value in commodity and logistics settings beyond what isolated node-level time-series models provide. They share, however, a limitation that matters for the question this thesis asks. Each is built around its own application-specific data representation; the representation is tailored to the task, not treated as a contribution. SupplyGraph is structured around production lines and demand centres in fast-moving consumer-goods setting; its node types, edge meanings, and temporal resolution make sense there but do not transfer directly to crude oil. The same point applies to the other applied papers. The methodological literature is largely silent on the upstream question of how to construct the data representation in the first place, taking it as given.

2.5 The gap

The three streams reviewed above are each useful for the questions they address, and none addresses the question this thesis poses. Official statistical balances publish monthly aggregates at fixed resolutions, with no canonical way to reconcile them across levels. The practitioner spreadsheet workflow produces operational balances quickly but buries inconsistency in unlabelled adjustment columns. Optimisation models work at a level of abstraction that leaves out the multi-resolution structure of the data. Graph and machine learning approaches take the upstream representation as given and tailor it to the task.

What is missing across all four is a representation that combines four properties at once: it admits multi-resolution data without ad-hoc reconciliation; it enforces single attribution at the schema level rather than through post-hoc checks; it preserves the provenance of derived quantities so that any value can be traced back to its sources; and it produces output that several different downstream consumers can read without modification. The thesis fills that gap by developing such a representation, implementing it on the U.S. crude oil network, and demonstrating that it produces materially more honest balances than the dominant current technique on the same input data.

Chapter 3

Domain background

This chapter provides the domain background needed to make the design choices in Chapters 4 and 5 comprehensible. It is deliberately focused: just enough description of the U.S. crude oil system to establish vocabulary, with the most attention given to the multi-resolution reporting structure that the framework responds to. Readers who want a full industry primer are referred to Stopford (2009) for the maritime side and to the EIA's published methodology documents for the statistical side.

3.1 The U.S. crude oil network

At the scale of the United States, the crude oil system can be described in terms of four physical layers: upstream production, midstream transport, storage, and downstream refining. The upstream layer brings crude to the surface. Onshore conventional production, offshore production concentrated in the federal waters of the Gulf of America (formerly the Gulf of Mexico), and shale and tight oil production drive U.S. output. The Permian Basin in Texas and New Mexico, the Bakken in North Dakota and Montana, and the Eagle Ford in Texas are the largest producing regions; together with offshore Gulf production, they account for the majority of U.S. crude.

The midstream layer moves crude from production regions to storage, refining centres, or export terminals. Gathering systems collect output from individual wells; trunk pipelines move crude over longer distances; marine transport handles international movements and, through the Jones Act fleet, certain domestic flows between PADDs; rail and truck handle smaller volumes where pipeline coverage is limited. The storage layer absorbs short-term imbalance. Commercial tank farms at major hubs — Cushing in Oklahoma, Patoka in Illinois, Houston, St. James, the LOOP system offshore Louisiana — provide the working storage used by refiners and traders. Alongside them sits the Strategic Petroleum Reserve, a set of four underground salt-cavern sites on the Gulf Coast operated by the federal government for emergency supply.

The downstream layer is where crude is converted into refined products. U.S. refining capacity is approximately 18 million barrels per day, with actual crude runs typically in the 15–17 million range. Capacity is heavily concentrated in PADD 3 on the Gulf Coast, which accounts for roughly half the U.S. total. For the purposes of this thesis the downstream layer marks the point at which crude exits the modelled system and reappears as refined products; the refining yield problem is in scope as a future extension but is not implemented in the active scenario.

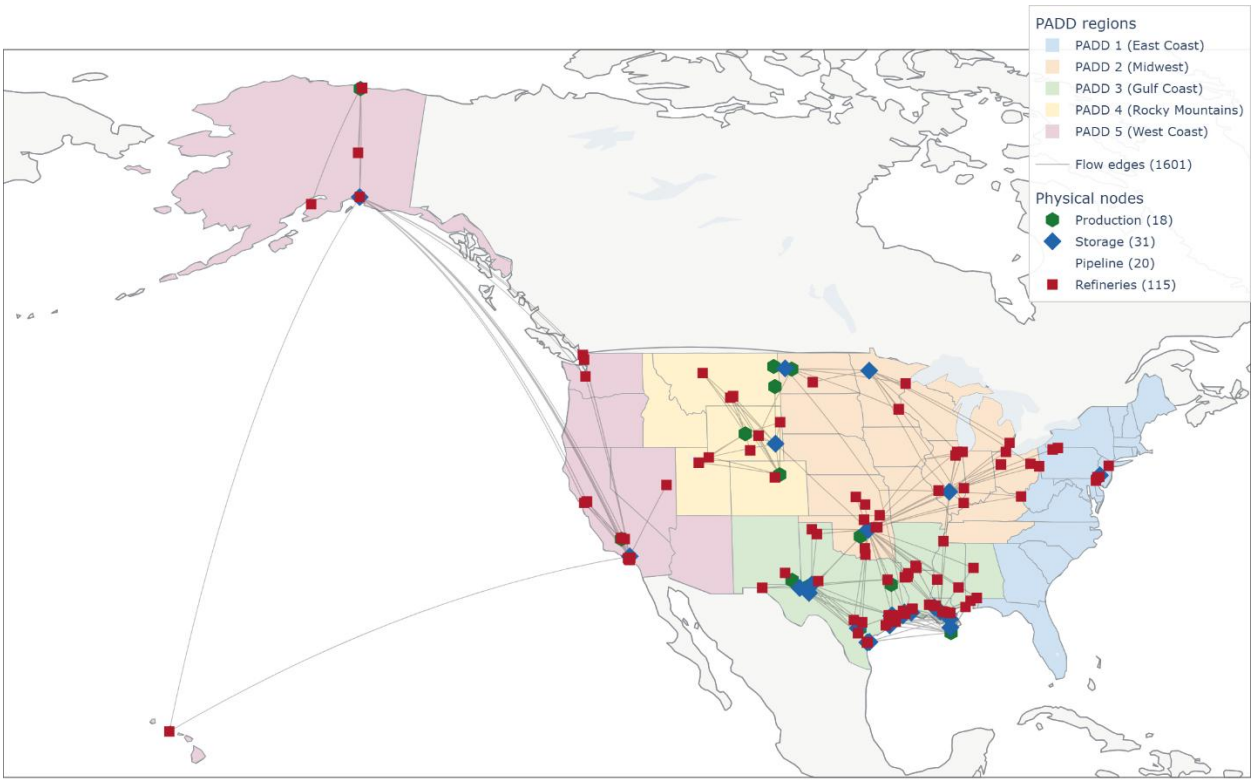

Figure 3.1. *U.S. crude oil transport network as instantiated in the framework. Production basins (green) feed gathering systems and origin terminals. Storage hubs (dark blue) sit at the major junctions; SPR sites (yellow) cluster on the Gulf coast. Refineries are shown in red, import and export terminals in orange. Latent per-edge flows from Permian to its three origin terminals are shown as grey dashed arrows. The PADD-3-to-PADD-5 in-transit corridor (Section 5.2) is highlighted in red.*

3.2 The multi-resolution reporting structure

The U.S. Energy Information Administration organises petroleum statistics around five Petroleum Administration for Defense Districts (PADDs). The districts were created in 1942 for wartime fuel rationing and have remained the dominant geographic structure for U.S. petroleum reporting ever since. PADD 1 covers the East Coast; PADD 2 the Midwest; PADD 3 the Gulf Coast; PADD 4 the Rocky Mountain region; PADD 5 the West Coast. Within each PADD, EIA reports refinery activity at the level of refining districts (PADD 3A, 3B, 3C, and so on) and production at the level of named basins and states.

The multi-resolution structure of EIA's publications is the central feature of the data that the framework must accommodate. The same physical quantity often appears at several resolutions at once, and the resolutions do not always agree. Table 3.1 summarises how the principal balance components are published.

Component	Resolution(s) published	Notes
Production	USA total, PADD, refining district, state, named basin	Same physical output reported at every level; named-basin sum within a PADD can differ from the PADD total by several hundred kbd.
Refinery runs	USA total, PADD, refining district	Facility-level data only for PADD 3 sub-districts.
Stocks	USA total, PADD, named hub (selected), SPR sites	Commercial, SPR, and in-transit reported separately at PADD level.
Imports	USA total, PADD, by country of origin	Per-PADD-per-country granularity; no per-terminal allocation.
Exports	USA total, PADD, by country of destination	Per-PADD-per-country granularity; no per-terminal allocation.
Inter-PADD flows	PADD pair (e.g. PADD 2 → PADD 3)	No per-pipeline allocation; many physical pipelines aggregated.

Table 3.1. *Principal balance components published by EIA at multiple resolutions. The same physical quantity is reported at several levels at once, with no canonical way to reconcile across levels.*

Three features of this structure matter for what follows. The first is that production at a named basin within a PADD does not in general add up to the published PADD total, because EIA includes some production in the PADD aggregate that is not separately named at the finer level. The second is that flows are reported at the level of PADD pairs rather than per pipeline, so the individual contribution of, say, the Capline or Diamond pipeline to PADD-2-to-PADD-3 movement is not publicly observed. The third is that several flows are physically real but not directly observed at any resolution — the per-terminal split of Permian outflows across Midland, Wink, and Crane is the canonical example. These three features are direct constraints on the representation.

3.3 What a representation must accommodate

The physical and reporting structure described above yields five requirements that the framework must meet:

- **Multi-resolution data without forced disaggregation.** Production observed at basin level, stocks at PADD level, consumption at refining-district level, and inter-PADD flows at PADD-pair level must coexist in a single representation. The framework must not require any of these to be "down-disaggregated" to a uniform finest-granularity view before being usable.
- **Junction fan-outs with unobserved per-edge splits.** The Permian outflow problem (one production node, three origin terminals, multiple pipelines from each, downstream destinations

observed only at aggregate level) is structural, not exceptional. Bakken gathering, the Cushing hub, and the Gulf Coast export complex all exhibit the same pattern at different scales.

- **Residuals as information, not noise.** The IEA convention of publishing the balancing item exists because the data does not close exactly, and the residual carries information about where the reporting system is failing to close. The framework must retain the residual as a first-class variable rather than absorbing it into adjacent flows.
- **Slow structural evolution.** The pipeline network changes deliberately over time — the Capline pipeline reversed direction in 2022, and a Bakken cross-state midstream pool came into operation in 2024. These are real operational events. The framework must accommodate them without requiring a wholesale graph rebuild every time data resolution improves.
- **A clear system boundary.** Foreign supply enters through Canadian and other import flows; U.S. crude leaves through export terminals to destinations that the framework does not track. The boundary needs to be explicit, with mass balance closing on the U.S. side without requiring the framework to model the foreign side.

These five requirements are not optional additions. They follow directly from the way the system operates and the way the data is published. The next chapter introduces the framework that responds to them.

Chapter 4

A graph-and-scenario framework

This chapter presents the representational framework as five design principles. Each principle is stated, explained in terms of what it enables, and shown to respond directly to one or more of the requirements identified at the end of Chapter 3. Together the five principles produce a representation in which every value traces to one source, aggregates close exactly to their constituents, mass balance holds at every physical node, and reporting residuals are labelled rather than hidden.

4.1 Physical infrastructure is orthogonal to data

> **Orthogonality.** Keep the physical layer and the data layer separate. Data updates must not change the graph; structural changes must not require data revisions.

The framework's central commitment is a separation between two layers that current techniques conflate.

The physical layer of the oil network changes slowly. New refineries open every few years; pipelines reverse direction every few decades (Capline reversed in 2022, Seaway in 2012); terminals are commissioned and retired on multi-year timescales. Each structural change reflects a real operational event in the physical world, often involving substantial capital expenditure, and is rare on the scale of a working scenario.

The data layer changes constantly, and at different rates for different variables. EIA publishes monthly series at the start of each month. Commercial providers update vessel positions every few minutes through AIS. Tank-level inventory at named hubs may be updated daily from satellite imagery; refinery operating status may be revised weekly. The granularity at which any given quantity is observed also varies: a flow that is reported today only at PADD-pair level may be reported tomorrow at per-pipeline level when a new commercial source becomes available.

Treating these two layers as a single thing forces every data improvement to rewrite the graph and every structural change to revisit the data. A workbook that incorporates a new finer-resolution series needs to modify the allocation logic that previously distributed the coarser figure; a graph that hard-codes the topology to match a particular dataset becomes obsolete when the dataset changes. The orthogonality principle resolves this by keeping the two layers separate. The physical graph is built once from public infrastructure data — corporate filings, FERC and PHMSA registers, OGJ refinery surveys, trade-press surveys — and updated only when a real physical event takes place. Data binds to that graph through per-

scenario assignments, which evolve continuously as new series and new sources become available, without ever modifying the underlying topology.

This orthogonality is the framework's primary contribution. The five principles that follow are operational consequences: each is necessary for the orthogonality to be workable in practice.

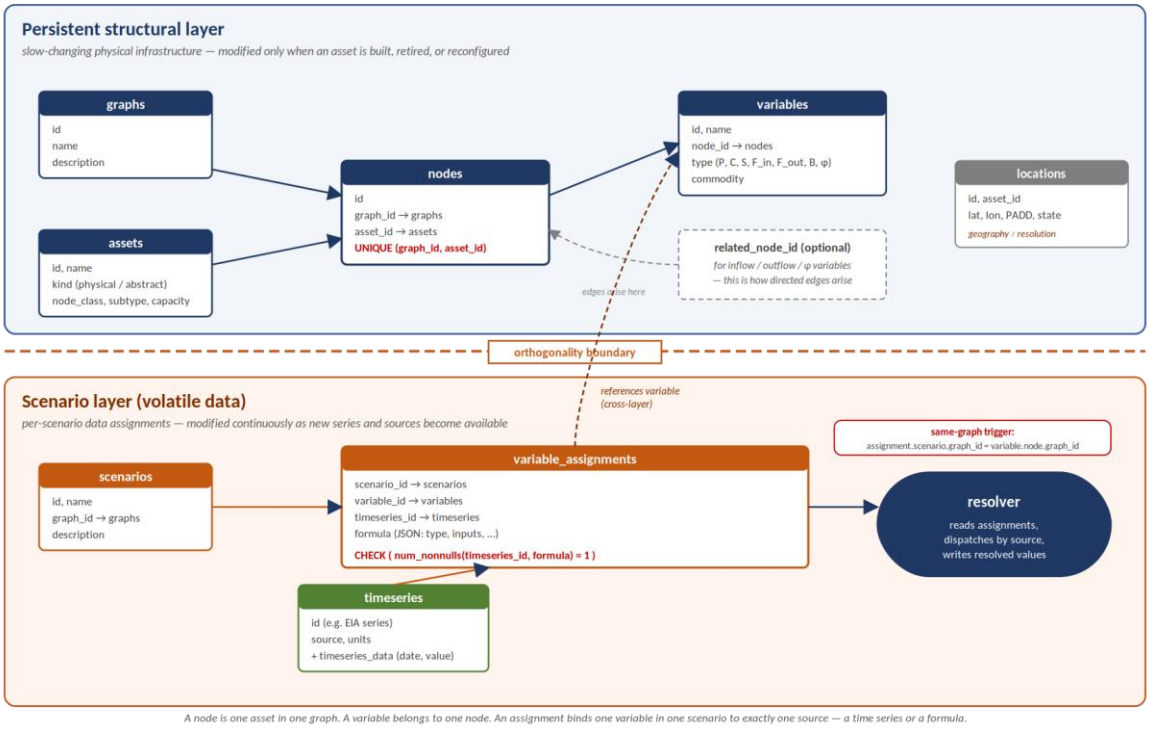

Figure 4.1. *The oil_network data model as enforced by the schema. The persistent structural layer (top, blue) holds graphs, assets, nodes, variables, and locations — the slow-changing physical infrastructure. The scenario layer (bottom, orange) holds scenarios, variable_assignments, and timeseries — the volatile data attached to that infrastructure. The CHECK constraint num_nonnulls(timeseries_id, formula) = 1 on variable_assignments enforces single attribution at the schema level. The same-graph trigger ensures that an assignment's scenario and its variable's node resolve to the same graph. The resolver consumes assignments and produces resolved values for downstream consumers.*

4.2 Assets are first-class nodes

> **Principle 1 — Assets as nodes.** Every physical asset — refinery, pipeline, vessel, terminal, basin, gathering system — is a node carrying its own inventory and mass balance. Edges are flows; assets are not edge attributes.

Every physical asset is represented as a node in the graph, with its own inventory variable and its own mass balance. Refineries, pipelines, vessels, terminals, basins, gathering systems, and storage hubs are all first-class nodes. Edges represent the directed movement of crude from one asset to another; they carry volume, but they do not hold it.

The alternative — a location-centric representation in which nodes denote places and assets sit on the edges between them — is common in the optimisation literature and well suited to strategic questions about where to locate capacity. It is unsuited to the question of where barrels actually are at any given moment. A trunk pipeline holds several days of throughput as line-fill, an amount comparable to a regional refinery's weekly runs; that line-fill is hidden state in a location-centric model, reconstructed indirectly from transit time and recent flows. A very large crude carrier holds two million barrels for the duration of its voyage; treating that volume as an edge attribute makes the cargo invisible to any analysis that asks where U.S. stocks actually sit. The asset-centric commitment exposes both: the pipeline's line-fill is its own stock variable, the vessel's cargo is its own stock variable, each accountable as the system makes them accountable in operational practice.

The same framing pays off when richer data sources are introduced. Genscape, now part of Wood Mackenzie, reports daily tank-by-tank inventory at the Cushing hub from infrared satellite imagery — roughly twenty individual storage tanks resolved rather than a single PADD-2 aggregate. Kpler tracks vessel cargoes globally through AIS feeds, resolving in-transit crude by ship, route, and grade. IIR Energy maintains a facility-level refinery operations database. In the asset-centric representation, each of these sources binds to the node it describes; in a location-centric representation, where pipelines and tanks are edge attributes, none of them would have a natural home.

4.3 Mass balance with an explicit balancing item

Principle 2 — Mass balance with B. Every node satisfies ΔS = P + ΣF_in − C − ΣF_out + B. The balancing item B is a labelled, first-class variable — never an unlabelled adjustment column.

Every physical node in the framework satisfies the mass-balance equation:

$$\Delta S = P + \Sigma F_{in} - C - \Sigma F_{out} + B$$

where ΔS denotes the change in stock, P is production at the node, C is consumption at the node, F_in and F_out are the inflows and outflows on the directed edges incident to the node, and B is the balancing item. The summations run over all neighbours of the node in the flow topology.

The balancing item B is a first-class variable on the node, not a cell in an unlabelled adjustment column. Its size and sign are themselves information. A persistently positive B at the USA aggregate level suggests that observed stock changes exceed what the reported flows would imply, which may point to under-reported exports, unreported imports, or timing mismatches. A sudden spike in B around a named disruption event — a hurricane, a pipeline outage, an SPR release — quantifies how much of the disruption the reporting system is failing to capture directly. The information content of B is not a side effect; it is the reason the variable exists. Chapter 5 uses this directly: B spikes around Hurricane Harvey, COVID-19,

the Colonial Pipeline shutdown, and the SPR release are visible in the framework and absent from the spreadsheet workflow that absorbs them silently.

Mass balance is enforced at physical assets only. Abstract aggregation views — PADD-level rollups, refining-district rollups, basin-level rollups — inherit mass balance as a consequence of being defined as sums over their physical constituents. They do not introduce a separate constraint of their own. Pass-through node types (pipelines, gathering systems, import and export terminals) have $P = C = \Delta S = 0$ by physical construction, so for those nodes the balance reduces to $\Sigma F_{in} = \Sigma F_{out}$, which is the conservation-of-flow constraint that network optimisation models naturally consume.

4.4 Observed or derived: single attribution at the schema level

> **Principle 3 — Observed or derived.** Every variable, in every scenario, is either observed (TS-bound) or derived (formula-bound) — never both.

For every variable on every node, the framework requires exactly one source of value. The variable is either observed — bound to a time series at the resolution where that series is published — or derived — computed from a formula over other variables. It cannot be both, and it cannot be neither.

Schema-level enforcement is stronger than post-hoc validation. An attempt to insert a row that is both time-series-bound and formula-bound, or neither, is rejected by the database before the data can propagate anywhere. Inconsistent state is not detected after the fact; it is unrepresentable. A complementary audit ensures that each time series under a given scenario is bound to exactly one variable, so that no source contributes to more than one place. Together, the two invariants give the framework its central consistency property: every value in the resolved output is traceable to exactly one source.

The constraint does not forbid declaring an aggregation relationship for cross-checking. A PADD-level production variable can be bound to its EIA series (the source of value) and also list its constituent basin and state variables in a separate aggregation-inputs column (the cross-check). The aggregation-inputs column is not a second source; it is the constraint set against which the observed aggregate can be compared. The view that materialises the comparison reports any gap between observed and constituents-sum at every observation date. PADD-level production is the canonical case: each PADD view's production variable is observed against the relevant EIA series and cross-checked against the sum of its named basins, with both sides agreeing within rounding tolerance.

4.5 Latent variables for unobserved flows

Principle 4 — Latent flows. Flows the data does not observe — junction splits, fan-outs, in-transit volumes — are declared as latent variables constrained by mass balance and capacity, not back-filled by ad-hoc allocation rules. Latent is not missing.

Some flows in the physical network are real but not directly observed at any resolution that public data provides. The most prominent example is the Permian Basin's outflow. Total Permian production is observed monthly through EIA's Texas and New Mexico series; receipts at the major Gulf Coast destinations are observed in aggregate; but the per-pipeline split between Midland, Wink, and Crane terminals, and from each of those terminals to the individual downstream pipelines (Gray Oak, Cactus II, EPIC, Wink-to-Webster), is not publicly reported. The same pattern recurs at the Bakken gathering system, at the Cushing hub, and at the Gulf Coast export complex.

The framework treats these unobserved per-edge flows as latent: declared variables with a clear structural role and a clear physical meaning, but without an assigned value. They appear in the resolved output with value = NULL and source = 'latent', which distinguishes them from values that are simply missing or unresolved through a bug. Mass balance at the junction still applies, so the latent variables are not unconstrained — their sum is fixed by the observed total at the upstream node, and their downstream destinations contribute additional constraints.

The alternative would be to fill these missing values with heuristics: an allocation in proportion to capacity, a historical average split, a rule of thumb from market knowledge. That approach makes the data look complete, but at the cost of blurring the distinction between measured information and modelling guesswork. A downstream consumer that reads the resolved output has no way to tell which numbers are observed and which are heuristic, which destroys the audit trail. The framework refuses that shortcut. A latent variable is unknown by design and is available as a decision variable to any downstream consumer — a linear programme that optimises flow allocation, a graph neural network that predicts unobserved values, a future commercial data source that binds them to observed values when the relevant information becomes available.

4.6 Multi-resolution data through aggregation views

Principle 5 — Abstract nodes do not have edges. Administrative aggregates (PADDs, refining districts, basin rollups) are abstract nodes over physical nodes, not duplicate storage. Mixed-resolution data feeds in through these views without double-counting.

The framework accommodates EIA's multi-resolution publications by binding each series at the level where it is published, not by spreading the value across constituents through allocation rules. PADD-level production is bound to the PADD-level node directly; basin-level production is bound to the basin-level

node directly; the relationship between them is declared as an aggregation relationship, not as a value-allocation rule. When both levels are observed simultaneously — as is typical for production — the comparison between observed parent and sum of observed children is a cross-check, not a reconciliation step.

This requires distinguishing between two kinds of relationship that the practitioner workflow tends to conflate. The first is a value relationship: variable A is the value source for some derived quantity B, in the sense that B's value is computed from A's value through a formula. The second is an aggregation relationship: variable A's value should equal the sum of A1, A2, A3 (or more generally, some formula over them) if both sides are observed correctly. The framework keeps these separate. A variable has exactly one value source (the schema-level CHECK of Section 4.4). It may additionally declare any number of aggregation relationships for cross-checking, none of which acts as a second source of value.

Observational aggregates — PADD views, refining-district views, basin-aggregate views — are abstract nodes that exist to host the published aggregate series. They have no flow edges of their own; their relationships to the physical layer are declared through aggregation-inputs references on their variables. This distinction also keeps geography metadata separate from the resolution hierarchy. Two refineries can share PADD 3 as a geography label without sitting in a parent-child structural relationship; the Permian-TX and Permian-NM nodes are in a parent-child relationship to the Permian basin aggregate, but they sit in different states for geography purposes. Labels are useful for filtering and presentation; they are not a substitute for the explicit aggregation declaration.

4.7 What the principles deliver together

Taken together, the five principles produce a representation with four properties. First, every value traces to exactly one source — either a named time series or a named formula over other variables, never both, never neither. Second, aggregate values close exactly to the sums of their constituents where both are observed, with any gap visible in a dedicated cross-check view. Third, mass balance holds at every physical node, by construction, with the residual carried in a named balancing item rather than absorbed into adjacent flows. Fourth, flows that are physically real but unobserved are labelled as latent and available to downstream consumers as decision variables. These four properties are the conditions under which an oil balance can be computed honestly from multi-resolution data, and they are the conditions the case study in the next chapter exploits.

Chapter 5

Case study: where the framework is better

The previous chapter set out the framework's design principles. This chapter compares the framework against the two dominant alternatives currently used to produce oil balances — the Excel spreadsheet workflow embedded in trading firms, consultancies, and government agencies, and the Python notebook workflow increasingly common on quantitative desks. The comparison is deliberately not about who produces a better single-month balance from the same data. On a fixed monthly EIA snapshot, all three methods, in the hands of a competent practitioner, arrive at approximately the same headline numbers. The interesting comparison is what happens when the world around the balance changes — when a new dimension is added to tracking, when a structural event breaks the assumptions the balance was built on, when a new data source becomes available, and when the analyst needs to verify that the balance is internally consistent.

5.1 Setup

Each of the four scenarios in this chapter is structured the same way. A one-sentence description of the change introduces what the desk is asked to do. Three method-narratives describe what each method does in response. A summary table compares the methods across five dimensions: what changes have to be made, how the analyst assesses the result, the rough order of analyst-hours consumed, the consistency risk level, and the kind of failure that occurs when the change is mishandled. The four scenarios in turn are: adding a grade dimension to basin production (§5.2); modelling a refinery turnaround or unplanned outage that breaks the monthly-average assumption (§5.3); receiving daily named-pipeline flow data from Genscape (§5.4); and finding inconsistencies after any of the above (§5.5). Section 5.6 aggregates the four scenarios into the headline claim.

The hours and risk levels reported in the tables are rough order-of-magnitude estimates drawn from operational experience with all three method-types in commodity-analyst settings, not the result of a controlled experiment. The point of the comparison is the order-of-magnitude gap between methods on each change, not the precision of any single figure. A reader from outside oil-market analysis should be able to follow the comparisons from the tables alone; the surrounding prose provides the operational detail that makes the gap concrete.

5.2 Adding grade dimensions to basin production

The desk decides to track grades produced at each basin. WTI from the Permian, sweet light from the Eagle Ford, Bakken Light, ANS heavy, medium sour from the Gulf — each should become a tracked quantity through production, transport, storage, and refining. Aggregate barrels stay the same; the data dimension widens from {asset} to {asset, grade}.

Excel spreadsheet. The existing workbook holds one column per balance line per PADD: production, runs, imports, exports, movements, stocks. Adding grades means each column duplicates by the grade dimension. Every formula must be edited. Reconciliation rules — every PADD-named-basin sum, every Jones Act in-transit allocation, every Cushing partition — must be redefined per grade. EIA per-grade

reporting is sparser than aggregate, so new residual cells multiply (one per grade per PADD per balance line). The analyst's grade-allocation assumptions become embedded in formula cells, undocumented. Two analysts working from the same source data produce different grade-resolved balances because their allocation choices differ, and the differences are not visible in the totals.

Pandas notebook. The DataFrame holding the balance is indexed by (date, PADD). Adding grades means widening the index to (date, PADD, grade), or pivoting to a wide format with grade-suffixed columns. Functions that transformed the aggregate balance must be rewritten to operate per grade. The new code can encapsulate allocation rules in functions rather than in cell formulas, which is more legible, but the rules themselves still have to be written and the grade definitions are bespoke to this notebook. Notebook state drift remains a risk: cells run out of order, intermediate state diverges. There is no schema-level enforcement, so a typo in a grade name silently creates a phantom grade rather than failing at the source.

Framework. The asset graph stays bit-identical. The 251 nodes, the 433 directed edges, the partition tree, and the capacity ceilings do not change. What changes is one row in the commodities table (a few INSERTs) and the variables table widens from 1,870 rows to approximately 1,870 × N rows. Each new variable is either bound to a per-grade observed series (where published) or inherits the aggregate-level formula with the per-(node, grade) residual carried as B. Partition closure operates per (node, grade) automatically; v_aggregation_consistency flags any divergence per (node, grade). Mass balance closes per (node, grade) at every physical asset. The single-attribution CHECK constraint on variable_assignments applies per (variable_type, commodity, node), so a typo in a grade name fails at INSERT — it is unrepresentable.

Figure 5.1 illustrates the asymmetry on a single asset — the Jones Act in-transit corridor `inter_padd_3_to_5_agg` — for visual clarity. The top panel shows each method's representation of the corridor at single-grade tracking, with the labelled stock in both columns. The bottom panel shows what happens when a grade dimension is added: the spreadsheet expands every column per grade and the in-transit row alone becomes a stack of per-grade residuals, while the framework's corridor node, edges, and capacity stay unchanged and only the variables widen.

Figure 5.1. Adding a grade dimension to the balance, illustrated on a single asset. Top panel: each method's representation of the Jones Act in-transit corridor at single-grade tracking — a row in a flat table (spreadsheet) versus a node with neighbours, capacity, flow variables, and mass balance (framework). Bottom panel: what each method does when a grade dimension is added — the spreadsheet rebuilds every column per grade, while the framework keeps the same nodes and edges and widens only the variables table.

	Excel spreadsheet	**Pandas notebook**	**Framework**
What changes	Cross-tab structure; every formula; reconciliation rules; source sheets	DataFrame schema; every transformation; reconciliation; data loaders	Add rows to commodities; data-layer pass widens variables table
How to assess	Manual sum-checks; eyeballing residuals	Assertion cells; ad-hoc tests	v_aggregation_consistency reports per-(node, grade) divergence
Effort (rough)	60–120 h	30–60 h	4–8 h
Consistency risk	High — silent disagreement between analysts	Medium — visible but bespoke	Low — schema-enforced
Failure mode	Bottom row reads zero; divergence buried in residuals	Notebook state drift; coverage gaps in assertions	Inconsistency unrepresentable for constrained patterns

Table 5.1. Adding a grade dimension to basin production: comparison across three methods. The framework's asset graph stays unchanged; only the variables table widens by the number of grades. The spreadsheet and the notebook rebuild to varying degrees and run a higher consistency risk.

5.3 Refinery turnaround or unplanned outage

A refinery enters an unplanned outage on day 5 of February and resumes operations on day 28. For twenty-three days the refinery's crude runs are zero; for the other five days they are at the design rate. The monthly EIA aggregate, which arrives at the end of March, reports the month-average runs — about 15% of nominal capacity. The desk needs to track the actual step-function in real time, because the question is not what February's average was; it is which Gulf Coast tank farms are filling up because their downstream offtake stopped, and whether crude vessels destined for the refinery were diverted to alternative receivers.

Excel spreadsheet. The existing workbook is structured around monthly EIA series. The cells assume monthly resolution: one row per month, one column per asset. To track the daily step-function, the analyst must either build a parallel daily sheet for the affected refinery and its upstream tank farms — manually reconciling the daily figures with the EIA monthly aggregate when it arrives at month-end — or abandon the daily detail and accept that the monthly figure represents an average that misrepresents what actually happened. The first option is operationally correct; the second is what most workbooks default to. If the analyst chooses the first, the propagation to upstream tank farms is also manual: an inventory-accumulation formula on each affected tank farm sheet, with the rate change happening at the day of the outage and reversing at the day of the restart. Capacity ceilings are not checked automatically.

Pandas notebook. The notebook holds the balance as a DataFrame indexed by date. Switching one asset to daily resolution while others remain monthly is technically straightforward — resample('M') aggregates the daily back to monthly, and joins between monthly and daily series can be done with pandas' alignment semantics. But the analyst still has to decide the resampling rule (mean? sum?), handle missing days (forward-fill? interpolation? raise an error?), and manage mixed-resolution joins where a monthly series is being compared against a daily-aggregated sum. The propagation logic must be written: if refinery X is offline for twenty-three days, what does that imply for tank farm Y's inventory? Mass balance is an assertion in code, not a structural property of the data model.

Framework. The timeseries_data table already carries observations at any granularity — daily, weekly, monthly observations coexist in the same table. For the refinery TAR, the analyst binds a daily IIR Energy series to the refinery's consumption variable for the duration of the outage; the resolver dispatches the daily observations directly. Aggregation to the monthly figure is handled by the resolver's mean-over-window rule. Mass balance at the refinery node holds at the daily resolution, so the resolver propagates the consumption drop to the upstream tank farm's inventory accumulation automatically through the partition closure on the tank farm's mass-balance equation. The capacity ceiling on the tank farm is asset_capacities.capacity_bbl, and v_capacity_violations reports any day on which the implied tank farm inventory exceeds capacity. The single-attribution CHECK on variable_assignments ensures the daily series and the monthly EIA aggregate are not both bound to the same variable in the same scenario. Hurricane Harvey, August 2017, provides a documented example. The storm made landfall on the Texas Gulf Coast on 25 August 2017 as a Category 4 hurricane, and approximately 25% of U.S. refining capacity was offline at peak for periods ranging from several days to over two weeks. The PADD 3 monthly EIA balance for August 2017 produced an implied residual of +193 kbd that the spreadsheet workflow absorbs into an unlabelled adjustment cell and the framework retains as a labelled balancing item B on the PADD 3 node. The residual is consistent with crude that left PADD 3 by tanker before the storm and was reported as exported, while the receiving inventories at downstream destinations had not yet

caught up; with delayed pipeline custody-transfer reports; and with refinery process losses that fell as throughput collapsed. Section 5.5 returns to this point with a table of comparable spikes around COVID-19 (March 2020), the Colonial Pipeline shutdown (May 2021), and the Strategic Petroleum Reserve release (May 2022 peak).

	Excel spreadsheet	**Pandas notebook**	**Framework**
What changes	New daily sheets; reconciliation rules; inventory propagation formulas per asset	DataFrame schema; resampling logic; propagation functions	One INSERT into variable_assignments binding a daily series
How to assess	Manual sum-checks; eyeball inventory trajectories	Assertion suite; resampling-consistency checks	v_node_balance_check; v_capacity_violations
Effort (rough)	40–80 h setup + 4–8 h per event	20–40 h setup + 2–4 h per event	2–4 h per event
Consistency risk	High — daily/monthly drift; analyst-specific reconciliation rules	Medium — visible but bespoke; mixed-resolution semantics drift	Low — mass balance enforced per node per day; capacity ceilings queryable
Failure mode	Inventory ceiling breaches go undetected; reconciliation rules diverge between analysts	False-negative assertions; resampling-rule choices undocumented	Inconsistency unrepresentable for constrained patterns

Table 5.2. Modelling a refinery turnaround or unplanned outage: comparison across three methods. The framework binds the daily series to an existing node; propagation to upstream assets is automatic via mass balance. The spreadsheet and the notebook require setup work per asset plus per-event effort, with consistency enforcement left to the analyst.

5.4 Receiving Genscape pipeline data

The desk subscribes to Wood Mackenzie's Genscape service, which reports daily flow rates on named pipelines via electromagnetic sensors mounted on the lines themselves. Approximately 91% of capacity connected to the Cushing hub is covered, including the major Permian outbound (Gray Oak, Cactus II, BridgeTex, Midland-to-ECHO), the Cushing connectors (Seaway, Spearhead, Pony Express), and the U.S.-Canada border pipelines (Enbridge Mainline, Keystone). EIA continues to report only aggregate inter-PADD movements; Genscape gives a daily breakdown for the named pipelines.

Excel spreadsheet. Adding Genscape data to a working EIA spreadsheet looks straightforward on first inspection — append columns, point the formulas at the new cells, recompute the balance — but the integration breaks on five distinct mismatches that the spreadsheet has no schema-level way to resolve. The frequencies disagree: Genscape reports daily, EIA monthly, and a pipeline that is down for four days mid-month leaves no trace in its monthly mean. The geographies disagree: Genscape names a pipeline (DAPL, Keystone), while EIA names a PADD pair, and a single EIA inter-PADD figure aggregates over an undisclosed set of pipelines, some of which Genscape names individually and some of which it does not. Double-counting becomes a live risk on every new column: if DAPL flows are appended as a separate series and the EIA PADD-2-to-PADD-3 movement series already incorporates them, every barrel on DAPL is now in the balance twice unless the analyst manually subtracts it from one side. Coverage is

incomplete: the nine percent of Cushing-connected capacity that Genscape does not monitor still shows up in the EIA aggregate, so a residual-pipeline column has to be maintained by hand each month. And the integration is not stable: every time Genscape adds coverage of a new pipeline (Seminole-Red, EPIC Crude, the latest reversed segment), the analyst must add a column, decide which existing aggregate it should be subtracted from, and update the residual definition. None of these decisions are encoded; they live in the analyst's head and in the workbook's commented cells, and they evolve as personnel rotate.

Pandas notebook. The notebook can ingest Genscape's API as a DataFrame and join with EIA aggregates. The structural advantages over Excel are real: the join logic is explicit code rather than hidden cell formulas; assertions can check that subtractions are applied; a function can encode the 'Genscape covers pipelines A, B, C; EIA aggregate is over A, B, C, D, E' mapping. But each new pipeline addition still requires updating the mapping function; the double-counting check is an assertion, not a structural property; mixed-resolution semantics (daily vs monthly) must be coded explicitly. As Genscape's coverage grows, the mapping function and the assertion suite must grow with it.

Framework. Genscape data binds to existing pipeline nodes. For each pipeline Genscape names (DAPL, Keystone, Gray Oak, Cactus II, …), the framework already has a node in the asset graph; the daily flow series binds to that node's outflow variable. The aggregation view ties Genscape-bound pipeline flows to the PADD-view inflow/outflow variables through the partition formula on the PADD-view, so the EIA inter-PADD aggregate is automatically reconciled against the sum of named pipeline flows. The 9% coverage gap emerges as the difference on the PADD-view node automatically, carried as B at the PADD-view. The schema-level CHECK on single attribution prevents the double-counting failure mode: the same variable cannot be bound to both the Genscape series and the EIA aggregate; one is the source of value, the other is the constraint set against which v_aggregation_consistency reports divergence. For new pipeline coverage additions: add a TS binding on the existing pipeline node; the aggregation view picks it up automatically; the residual on the PADD-view rebalances automatically; no other code changes are required.

	Excel spreadsheet	**Pandas notebook**	**Framework**
What changes	Columns per pipeline; manual subtractions; residual definition	Join code; pipeline-to-aggregate mapping function; assertions	N INSERTs into variable_assignments (one per Genscape-named pipeline)
How to assess	Manual sum-checks; eyeball Genscape totals against EIA	Assertion suite; plots of Genscape sum vs EIA aggregate	v_aggregation_consistency reports divergence per (PADD, month)
Effort initial	20–40 h	15–30 h	N × 1–2 h
Effort per new pipeline	2–4 h	1–3 h	1–2 h
Consistency risk	High — silent double-counting; residual-definition drift	Medium — bespoke assertions; mapping-function staleness	Low — schema-enforced single attribution; coverage gap auto-flagged

Table 5.3. Integrating Genscape daily pipeline-flow data on top of EIA monthly aggregates: comparison across three methods. The framework binds each Genscape series to an existing pipeline node; the aggregation view reconciles automatically, and double-counting is unrepresentable. The spreadsheet and the notebook accumulate per-pipeline integration debt.

5.5 Finding inconsistencies

After a month-end refresh of the balance — whether from monthly EIA, daily Genscape, or any combination — the desk needs to verify that the balance is internally consistent. Did all PADD-named-basin sums tie to PADD totals? Did all pipeline flows balance against inter-PADD aggregates? Did inventory changes match flow differences? Did any flow exceed its asset capacity? Did any variable get double-bound to two conflicting series? These are not exotic questions; they are the daily verification a desk does before relying on the balance.

Excel spreadsheet. Inconsistency-finding in a spreadsheet is by manual inspection. The analyst writes one or more check sheets where critical sums are recomputed and the difference is displayed in a corner cell, often colour-coded if it exceeds a tolerance. The coverage is whatever the analyst remembers to check. A typical sequence is: PADD totals against named-basin sums; monthly stock changes against (production − consumption + imports − exports + movements_in − movements_out); refinery runs against the EIA refining-district aggregate. Outside the analyst's check sheets, inconsistencies are silent. When the workbook is handed over to a new analyst, the check coverage walks away with the previous one.

Pandas notebook. Inconsistency-finding in a notebook is more systematic. The analyst writes assertion cells: assert (padd_sum == padd_total).all(); assert (stock_change == flows.sum()).all(). These assertions can be packaged as a test suite and run automatically on each balance refresh. Coverage is whatever the analyst writes — better than a spreadsheet's manual sheets but still bespoke. The Achilles' heel of assertion-based testing is false negatives: an assertion that's never run never catches anything. As the scenarios evolve — new grades, new pipelines, new outage models — the assertion suite must be maintained to keep pace. Notebook state drift means the same cell can produce different assertion outcomes across re-runs.

Framework. Inconsistency-finding in the framework runs in two layers: schema-level enforcement at write time, and view-level audit at read time. The schema layer carries the CHECK constraint on num_nonnulls(timeseries_id, formula) = 1 on variable_assignments, foreign-key constraints across the asset graph, and trigger-enforced same-graph rules. The view layer materialises three consistency audits that re-run automatically when the resolver completes:

v_aggregation_consistency reports any (node, variable_type) pair where the observed parent value disagrees with the sum of partition children, per observation date, with the magnitude of the disagreement quantified. v_resolution_anomalies reports resolver-level oddities — long forward-fill runs, negative derived values, partial dispatch (some children observed, some latent) — with severity ranked. v_capacity_violations reports any flow that exceeds the receiving or originating asset's capacity_bd ceiling.

The schema enforcement is part of the database; the audit views are part of the resolver pipeline. Neither requires per-scenario setup. New grades, new pipelines, new outage observations — all are checked automatically by the same views without any change to the audit infrastructure. The framework's B series at the USA aggregate level also acts as a passive monitor of where the reporting system is and is not closing cleanly. Table 5.5 reports the absolute value of B at the USA aggregate around four named disruption events, alongside the ten-year median for comparison.

	Excel spreadsheet	**Pandas notebook**	**Framework**
What changes per scenario	New check sheets; tolerance tuning	New assertion cells; suite maintenance	Nothing — built into schema + resolver pipeline
How to assess	Read check sheets visually	Run test suite; inspect failures	Query v_aggregation_consistency, v_resolution_anomalies, v_capacity_violations
Effort initial	5–20 h	10–30 h	0 h (built-in)

Effort per scenario change	1–2 h per month	2–5 h per scenario	0 h
Consistency risk	High — silent coverage gaps; analyst-specific tolerances	Medium — false-negative assertions; coverage gaps where assertions are missing	Low — schema-level CHECKs + automatic audit views

Table 5.4. Finding inconsistencies after a balance refresh: comparison across three methods. The framework's consistency-audit infrastructure is built into the schema and the resolver pipeline; it requires zero per-scenario setup. The spreadsheet and the notebook require per-scenario test infrastructure that the analyst maintains.

Event	Month	|B| at USA agg (kbd)	Multiple of median
Median, all months 2015–2024 (baseline)	—	~260	1.0×
Hurricane Harvey	Aug 2017	~145	0.6×
COVID-19 demand collapse	Mar 2020	~748	2.9×
Colonial Pipeline shutdown	May 2021	~596	2.3×
Strategic Petroleum Reserve release	May 2022 (peak)	~427	1.7×

Table 5.5. Absolute value of the balancing item B at the USA aggregate around named disruption events, compared with the ten-year median. The framework retains B as a first-class labelled variable, so spikes around named events are visible in the resolved output without bespoke audit code. The spreadsheet workflow absorbs the same residual into an unlabelled adjustment cell. Hurricane Harvey at the USA aggregate is the partial exception: its national signal is partly cancelled by offsetting flows between PADD 3 and the rest of the country, while the per-PADD signal documented in §5.3 remains clear.

5.6 What the comparison establishes

The four scenarios establish a consistent pattern. The cost of evolving the balance, measured in analyst-hours, differs by one or two orders of magnitude between the framework and either of the alternatives, summarised in Table 5.6.

What drives the gap is not technical sophistication. A competent Python notebook can do everything a spreadsheet can do and more — modular code, version control, programmatic assertions, mixed-resolution joins. The notebook's advantage over the spreadsheet on every scenario is genuine. The framework's advantage over the notebook is genuine in turn, but it has a different character: it is structural rather than operational. The framework enforces single attribution, partition closure, and capacity constraints at the database level, before any code runs. The notebook can replicate these checks as assertions, but the assertions are bespoke per scenario, depend on the analyst remembering to write them, and can be silently bypassed when the notebook is re-run with stale state.

The cost asymmetry compounds because the four scenarios are rarely encountered in isolation. A real desk does all four changes in a single quarter — adds grade tracking for a specific basin, models a Gulf Coast TAR, integrates Genscape coverage on the Permian outbound complex, and runs consistency audits monthly. The framework's setup cost is one-time and structural; the alternatives' setup cost is per-scenario and additive. Over a year of operational use, the gap is not 60–120 hours versus 4–8 hours on grades; it is the sum across all scenarios that the desk encounters, with consistency risk compounding silently in the alternatives.

The framework is not a methodologically more sophisticated balance — it produces the same monthly aggregate values from the same EIA series in steady state. It is a representation that absorbs change without requiring rebuild, and that enforces consistency at the schema level rather than relying on analyst vigilance. The thesis claim is that this is the structural property a representation should have for production use in a setting where balances evolve continuously and consistency failures are operationally costly.

Scenario	Excel spreadsheet	Pandas notebook	Framework
1. Adding grade dimensions to basin production	60–120 h	30–60 h	4–8 h
2. Refinery TAR / unplanned outage	40–80 h + 4–8 h per event	20–40 h + 2–4 h per event	2–4 h per event
3. Receiving Genscape pipeline data	20–40 h initial; 2–4 h per pipeline added	15–30 h initial; 1–3 h per pipeline added	1–2 h per pipeline added
4. Finding inconsistencies	5–20 h initial; 1–2 h per month	10–30 h initial; 2–5 h per scenario	0 h (built-in)

Table 5.6. Aggregate cost asymmetry across the four change-management scenarios. Hours are rough order-of-magnitude estimates drawn from operational experience. The point of the table is the order-of-magnitude gap between methods, not the precision of any single figure. The framework's setup cost is one-time and structural; the alternatives' setup cost is per-scenario and additive over the operational lifetime of the balance.

32

33

Conclusion and future work

6.1 Contribution

This thesis has developed a representational framework for crude oil logistics data that treats physical infrastructure as orthogonal to the data attached to it. The framework rests on five principles: physical infrastructure is a slowly evolving graph separate from fast-moving data assignments; assets are first-class nodes carrying their own inventory and mass balance; mass balance includes an explicit labelled balancing item rather than an unlabelled adjustment column; every variable has exactly one source of value, enforced at the schema level; and flows that are real but unobserved are labelled latent rather than filled with heuristics.

The framework has been implemented as a working system for the U.S. crude oil network covering January 2015 to December 2024 at monthly resolution, with 251 assets, 433 directed flow edges, and 1,870 variables under the active scenario. The implementation is described in Annex A. The framework's output has been compared head-to-head against the dominant current technique — the spreadsheet workflow that produces operational oil balances inside trading firms, consultancies, and integrated oil companies — on the same input data. The comparison shows that the framework produces a materially different and more honest output: persistent structural residuals like the six-million-barrel PADD 5 in-transit volume

land on named nodes rather than in unallocated adjustment cells, and the balancing item retains its information content around disruption events that the spreadsheet workflow absorbs silently.

6.2 Limitations

Three limitations of the work as presented should be flagged. First, the implemented instance treats crude oil as a single commodity. The schema admits per-grade decomposition by treating commodity as a further dimension on variables, and a registry of nineteen named U.S. grades is seeded, but the resolver currently propagates balances at the single-commodity level. Extending it to per-grade balances is the natural next step on the implementation side.

Second, the framework distinguishes labelled-latent flows from labelled-balancing items, but it does not by itself interpret B. A spike in B around Hurricane Harvey is visible to any downstream consumer that reads the variable, but the framework does not by itself diagnose whether the spike reflects under-reported exports, delayed custody transfer, refinery process losses, or some combination. That diagnosis is downstream work, and depends on additional context that the framework does not currently host.

Third, the implementation is a research artefact rather than a production system. Issues such as deployment, monitoring, access control, and operational scaling are outside the scope of the thesis. The framework's design does not preclude any of these, but turning the artefact into a production deployment is engineering work that this thesis does not undertake.

6.3 Future work

Three lines of future work follow naturally from the framework as it stands.

Per-grade decomposition. With the commodity dimension already on variables and the grade registry seeded, the resolver can be extended to propagate per-grade balances at each tank or refinery. Refinery yield modelling — expressing refined-product output as a derived variable through formula inputs, with grade-specific coefficients — is a natural extension of the same machinery. The framework's properties extend per-grade without structural change: every per-grade variable would carry its own assignment, mass balance per grade would hold at each node, and per-grade latent flows would be available to downstream consumers in the same form as the single-commodity latents are today.

Graph neural network forecasting. The framework's resolved output is structurally what the temporal graph neural network architectures most relevant to logistics networks — DCRNN, STGCN, TGN, TGAT — consume as input: a node-by-time feature matrix, a fixed adjacency, and a conservation constraint that can be enforced as an architectural prior, a soft loss penalty, or a hard projection. The modelling work required to put a forecasting model on top of the framework is substantial — architecture selection,

train/validation/test design that does not leak across time, target selection, baseline comparison — but the data layer is ready. This is the line of work this framework is most directly designed to enable.

Other commodity networks. The orthogonality of physical infrastructure and data is not specific to crude oil. Natural gas networks, electricity grids, and refined-product distribution systems share the same structural features: a slowly evolving physical layer, multi-resolution observational data, junctions with unobserved per-edge splits, and reporting residuals that carry information. The U.S. crude oil instance built for this thesis is a template, not a destination; the same five principles and the same schema design support analogous instances for these other networks with only minor adjustments to the variable types and the node taxonomy.

6.4 Closing remarks

The crude oil logistics system has been studied for decades and modelled in many ways. This thesis does not claim to add to the operational understanding of the system itself. What it adds is a representation of that system that admits the multi-resolution structure of the public data, enforces consistency at the schema level, preserves the provenance of derived quantities, and produces an output in which the residuals that current techniques hide are instead labelled and traceable.

The framework is a substrate, not a destination. Its value will be measured not by the principles it states but by the downstream work it makes easier: the forecasting models that consume its output, the optimisation tasks that take its latents as decision variables, the price models that read its balancing items as signal, the audits that follow its provenance back to source. The case study in Chapter 5 demonstrates the immediate gain — a balance that does not throw away the information the data already contains. Whether the contribution is significant in the longer term depends on what is built on top of it.

References

Ahn, S., Yoon, J., & Park, J. (2024). A probabilistic graph neural network for supply and inventory prediction with attention-based architectures. International Journal of Production Research, 62(15), 5234–5251.

Bornstein, G., Krusell, P., & Rebelo, S. (2017). Lags, costs, and shocks: An equilibrium model of the oil industry. NBER Working Paper 23423. Cambridge, MA: National Bureau of Economic Research.

Fattouh, B. (2011). An anatomy of the crude oil pricing system. Oxford Institute for Energy Studies WPM 40.

Fernandes, L. J., Relvas, S., & Barbosa-Póvoa, A. P. (2013). Strategic network design of downstream petroleum supply chains: Single versus multi-entity participation. Chemical Engineering Research and Design, 91(8), 1557–1587.

Ghaithan, A. M., Attia, A., & Duffuaa, S. O. (2017). Multi-objective optimization model for a downstream oil and gas supply chain. Applied Mathematical Modelling, 52, 689–708.

Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 855–864.

Hamilton, J. D. (2009). Understanding crude oil prices. The Energy Journal, 30(2), 179–206.

Hamilton, W. L., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. Advances in Neural Information Processing Systems, 30, 1024–1034.

International Energy Agency. (2024). Oil Market Report. Paris: IEA. Methodology notes accessed via https://www.iea.org/reports/oil-market-report.

Jin, M., Koh, H. Y., Wen, Q., Zambon, D., Alippi, C., Webb, G. I., King, I., & Pan, S. (2024). A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 46(12), 10466–10485.

Joly, M., Moro, L. F. L., & Pinto, J. M. (2002). Planning and scheduling for petroleum refineries using mathematical programming. Brazilian Journal of Chemical Engineering, 19(2), 207–228.

Kazemi, Y., & Szmerekovsky, J. (2015). Modeling downstream petroleum supply chain: The importance of multi-mode transportation to strategic planning. Transportation Research Part E: Logistics and Transportation Review, 83, 111–125.

Kilian, L., & Murphy, D. P. (2014). The role of inventories and speculative trading in the global market for crude oil. Journal of Applied Econometrics, 29(3), 454–478.

Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. Proceedings of the 5th International Conference on Learning Representations.

Kosasih, E. E., & Brintrup, A. (2022). A machine learning approach for predicting hidden links in supply chain with graph neural networks. International Journal of Production Research, 60(17), 5380–5393.

Li, Y., Yu, R., Shahabi, C., & Liu, Y. (2018). Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. Proceedings of the 6th International Conference on Learning Representations.

Lima, C., Relvas, S., & Barbosa-Póvoa, A. P. (2016). Downstream oil supply chain management: A critical review and future directions. Computers & Chemical Engineering, 92, 78–92.

Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 701–710.

Pindyck, R. S. (2001). The dynamics of commodity spot and futures markets: A primer. The Energy Journal, 22(3), 1–29.

Pinto, J. M., Joly, M., & Moro, L. F. L. (2000). Planning and scheduling models for refinery operations. Computers & Chemical Engineering, 24(9–10), 2259–2276.

Ribas, G. P., Hamacher, S., & Street, A. (2010). Optimization under uncertainty of the integrated oil supply chain using stochastic and robust programming. International Transactions in Operational Research, 17(6), 777–796.

Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Bronstein, M., & Monti, F. (2020). Temporal graph networks for deep learning on dynamic graphs. ICML 2020 Workshop on Graph Representation Learning.

Stopford, M. (2009). Maritime Economics (3rd ed.). London: Routledge.

U.S. Energy Information Administration. (2024). What drives crude oil prices: Balance. Washington, DC: U.S. Department of Energy. https://www.eia.gov/finance/markets/crudeoil/balance.php

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. Proceedings of the 6th International Conference on Learning Representations.

Wasi, A. T., Islam, T., Akib, A. R., & Bappy, M. (2024). SupplyGraph: A benchmark dataset for supply chain planning using graph neural networks. AAAI 2024 Workshop on Graph Learning for Industrial Applications.

Working, H. (1949). The theory of price of storage. American Economic Review, 39(6), 1254–1262.

Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2020). A comprehensive survey on graph neural networks. IEEE Transactions on Neural Networks and Learning Systems, 32(1), 4–24.

Xu, D., Ruan, C., Korpeoglu, E., Kumar, S., & Achan, K. (2020). Inductive representation learning on temporal graphs. Proceedings of the 8th International Conference on Learning Representations.

Yu, B., Yin, H., & Zhu, Z. (2018). Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. Proceedings of the 27th International Joint Conference on Artificial Intelligence, 3634–3640.

Annex A

Implementation

This annex documents the implementation of the framework described in Chapter 4. It is reference material for readers who want enough detail to reproduce, audit, or extend the working system. The implementation is a research artefact built with PostgreSQL, Python, and a set of materialised SQL views; it is not a production system.

A.1 Schema

The framework is implemented as a relational schema in PostgreSQL, named oil_network. Four primary entities support the design principles of Chapter 4.

assets is the persistent registry of physical entities. Each row carries an asset identifier, a kind flag (physical or abstract), a node class (production, infrastructure, observational), a subtype (refinery, gathering, pipeline, region_view, and so on), and capacity attributes where relevant.

nodes represents an asset's appearance in a particular graph. Each node binds to exactly one asset and to one graph, with a UNIQUE constraint on (graph_id, asset_id). The starter scenario uses a single graph_id, so the assets-to-nodes correspondence is one-to-one in the current instance, but the structure admits multiple graphs over the same asset set (a per-grade view, a per-operator view, a North American extension that adds Canadian and Mexican assets).

variables carries the data attributes of nodes. Each variable has a type (production, consumption, inventory, balancing item, inflow, outflow), a commodity (crude, in the active scenario), an optional related_node_id (for inflow and outflow variables that reference another node), and a name that serves as a stable audit identifier.

variable_assignments binds each variable to a value source under a given scenario. The CHECK constraint of Section 4.4 sits on this table: CHECK (num_nonnulls(timeseries_id, formula) = 1). An attempt to insert an assignment that is both time-series-bound and formula-bound, or neither, is rejected by the database.

Geographic metadata is kept in a separate oil_network.locations table to enforce the separation between geography and the resolution hierarchy that Section 4.6 requires. Same-graph triggers ensure that an assignment's scenario and its variable's node both resolve to the same graph, so that scenario state cannot leak across graphs.

A.2 The starter U.S. crude oil graph

The active scenario, starter_us_crude_2015_2025, instantiates the U.S. crude oil network at monthly resolution from January 2015 to December 2024 inclusive. Table A.1 summarises the node inventory by class.

Class	Count	Examples
Physical — production	18	Permian-TX, Permian-NM, Bakken-ND, Bakken-MT, Eagle Ford-TX, Gulf of America offshore, state-conventional fields, per-PADD residuals
Physical — gathering	6	Permian-TX, Permian-NM, Bakken-ND, Bakken-MT, Eagle Ford, ANS gathering systems
Physical — origin terminal	6	Midland, Wink, Crane, Johnson's Corner, Three Rivers, Valdez
Physical — storage hub	10	Cushing (with three operator sub-terminals), Patoka, Houston, St. James, LOOP, others
Physical — SPR site	4	Bayou Choctaw, Big Hill, Bryan Mound, West Hackberry
Physical — refinery	115	19 named refineries, 5 PADD residuals, sub-aggregates totalling 24 active facilities
Physical — pipeline	37	20 named U.S.-internal lines, 4 cross-border, 12 inter-PADD aggregates, Bakken cross-state connector
Physical — import/export terminal	19	Ingleside, Houston complex, Nederland, LOOP, per-PADD import and export aggregates
Physical — boundary	3	Foreign supply, Canadian oil sands, foreign export destination
Abstract — observational	33	USA view, PADD views, refining-district views, state views, basin aggregates, stock-decomposition aggregates

Table A.1. *Node inventory under the active scenario, by class and subtype. Physical nodes carry geographic coordinates, capacity, and flow edges. Abstract nodes are aggregation views without flow edges.*

The asset graph was constructed from public domain knowledge (corporate filings, FERC and PHMSA registers, OGJ refinery surveys, Global Energy Monitor pipeline trackers) rather than from what EIA happened to publish. The result is an asset graph that covers approximately 100% of U.S. modelled refining capacity (17,484 kbd against the real system's roughly 17,500 kbd) and 113% of production capacity at peak (slightly more than the real system because some assets are at headroom rather than steady-state utilisation). EIA time series then bind into this superset where the data is available; missing series leave the corresponding variables latent or formula-derived rather than forcing the asset graph to shrink.

The graph contains 433 directed flow edges and 1,870 variables under the active scenario. Of these, 90 variables are time-series-bound (observed), distributed across U.S. and PADD-level production, refinery runs, stocks, imports, exports, and inter-PADD movements. The remaining 1,780 variables resolve through formulas — structural zeros for pass-through node types, aliases for derived aggregates, sums for partition rollups, arithmetic combinations for residual definitions, or latent flags for unobserved per-edge junction flows.

A.3 The resolver

The resolver is a Python script (resolve_scenario.py) that reads variable assignments for a given scenario, applies a dispatch loop over the variable set, and writes a per-(scenario, variable, observation date) value table. The dispatch rules are applied in priority order:

- **Observed:** if the assignment is time-series-bound, look up the most recent value at or before the observation date (last-observation-carried-forward) and write it with source='observed'.
- **Structural zero:** if the node type forces the variable to zero (e.g. production on a pass-through pipeline), write 0 with source='zero'.
- **Latent:** if the formula resolves to latent(), write NULL with source='latent'. Includes reverse-mirror promotion: if a flow variable is latent on one direction but the counterpart on the reverse direction is observed, the counterpart's value propagates.
- **Sum, arithmetic: evaluate the formula over its inputs in topological order and write the result with source naming the rule applied.**
- **Closure:** rarely used; resolves a residual from its complement plus a published total.
- **Unresolved:** the fallback. In a healthy scenario this rule is never triggered, and the runtime audit fails if any variable lands here.

The resolver runs in approximately 20 seconds on a developer laptop for the full ten-year scenario. The dispatch counts for the active scenario are: 90 observed, 542 structural zero, 767 latent, 451 arithmetic, 5 sum, 15 reverse-mirror, 0 closure, 0 unresolved. The values are recomputed at every run; the audit trail (scenario_html_artefacts, scenario_resolved_values) records the run identifier and timestamp.

A.4 Consistency properties demonstrated on the live system

The framework's four consistency claims (Section 4.7) are demonstrated by queries against the layered views. Table A.2 reports the result of each audit on the live system at the time of writing.

Claim	Audit query	Result on the live system
Single attribution	SELECT timeseries_id, COUNT(*) FROM variable_assignments WHERE scenario_id = active GROUP BY timeseries_id HAVING COUNT(*) > 1	0 rows returned (90 time series → 90 variables, strict 1:1)
Partition closure	SELECT * FROM v_node_balance_check WHERE gap > tolerance	0 rows returned at every PADD aggregate and at USA, every observation date
Mass balance	SELECT * FROM v_node_balance WHERE residual > tolerance AT physical nodes	0 rows returned at all 217 physical nodes, every observation date
B around named events	SELECT observation_date, ABS(value) FROM scenario_resolved_values WHERE variable = usa_view.balancing_item ORDER BY ABS(value) DESC LIMIT 10	Top 10 absolute values cluster on Aug 2017, Apr–Jun 2020, May 2021, 2022 SPR-release months

Table A.2. *Consistency audits and their results on the live system. The four claims of Section 4.7 are all met.*

These results are reproducible: any reader with access to the implementation can re-run the audit queries and verify the same outcomes. The framework's claim to be a contribution rests on this reproducibility, not on assertion.

Annex B

Variable catalogue and starter scenario inventory

This annex provides reference tables for the variables and assignments in the active scenario starter_us_crude_2015_2025.

B.1 Variable types

Type	Symbol	Description	Relational
production	P	Crude produced at this node	no
consumption	C	Crude consumed at this node (refinery runs)	no
inventory	S	Crude stocks held at this node	no
balancing_item	B	Reporting residual on this node	no
inflow	F_in	Crude flowing in from a specific related node	yes
outflow	F_out	Crude flowing out to a specific related node	yes

Table B.1. *Variable types in the active scenario. Relational variables carry a related_node_id identifying the counterpart in the directed flow.*

B.2 Authoritative bindings by subsystem

Subsystem	Series count	Examples of EIA series bound
USA aggregates	9	MCRFPUS2 (production), MCRRIUS2 (refinery runs), MCRSTUS1 (stocks), MCRIMUS2 (imports), MCREEXUS2 (exports), MCRUA (balancing item)
PADD-level series	30	MCRFPP{1..5} (production by PADD), MCRRIP{1..5} (refinery runs), MCESTP{1..5} (stocks), MCRMP_{i}_{j} (inter-PADD movements)
Stock decomposition	10	MCRSFP{N}1 (tank farms and pipelines per PADD), MCRRSP{N}1 (refinery stocks per PADD), per-SPR-site inventory
Production by state/basin	15	MCRFPUSP3TX (Texas-Permian), MCRFPUSP2ND (North Dakota Bakken), MCRFPUSGOM (Gulf of America), STEO basin forecasts
Refining districts	10	MCRRIP3{A,B,C} and other refining-district refinery runs

Subsystem	Series count	Examples of EIA series bound
Import/export by country	16	Per-PADD foreign and Canadian import series, per-PADD export series by destination region

Table B.2. *Authoritative time series bindings by subsystem under the active scenario. 90 distinct EIA series bind to 90 distinct variables in strict one-to-one attribution.*

B.3 Latent variables by location

Junction	Latent flow count	Physical interpretation
Permian outbound	9	permian_tx → three origin terminals; each origin terminal → multiple downstream pipelines
Bakken gathering	8	bakken_nd_gathering and bakken_mt_gathering → multiple downstream pipelines and rail outlets
Cushing hub	6	Three operator sub-terminals (Enterprise, Plains, Magellan), per-direction pipeline interfaces
Gulf export complex	12	Per-terminal flows to foreign_export_destination; aggregate observed at PADD-3 level
Inter-PADD pipelines	24	Per-pipeline allocation within each inter-PADD corridor; aggregate observed at PADD-pair level
Bakken cross-state	4	Bidirectional pipe_bakken_xstate connector, all four flow variables currently latent

Table B.3. *Latent variables under the active scenario, by junction. All are real flows under physical constraints whose per-edge values are not publicly observed.*

Annex C

Graph neural networks: a primer

This annex provides a self-contained technical introduction to the graph neural network architectures referenced in Chapter 2 and Section 6.3 as the natural next downstream consumer of the framework. Readers who want a deeper methodological treatment are referred to the surveys by Wu et al. (2020) and Jin et al. (2024).

C.1 Node embeddings and message passing

A graph neural network learns node representations through iterative neighbourhood aggregation. Each node starts with its own feature vector — in the crude oil case, a vector of operational variables such as production, consumption, inventory, and inflow/outflow rates — and repeatedly updates that representation by pulling in information from its neighbours. Gilmer et al. (2017) showed that the dominant architectures (GCN, GAT, GraphSAGE) are all special cases of the same three-step mechanism applied layer by layer: send (each node sends a transformed copy of its feature vector to its neighbours), aggregate (each node collects all incoming messages and combines them into a neighbourhood summary), and update (each node combines the summary with its own representation to produce a new embedding).

After one round, each node knows about its immediate neighbours. After two rounds, it knows about nodes two hops away. After k rounds, information has propagated across k-hop neighbourhoods — which corresponds, in the crude oil case, to how a physical disruption propagates through a pipeline network.

C.2 Graph convolutional networks (GCN)

GCN (Kipf and Welling, 2017) is a mathematically principled formalisation of message passing, derived from spectral graph theory through two simplifications. The GCN layer formula is:

$$H^{(l+1)} = \sigma(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)})$$

where $\tilde{A} = A + I$ is the adjacency matrix with self-loops added, $\tilde{D}$ is the corresponding degree matrix, $H^{(l)}$ is the matrix of node representations at layer ℓ, $W^{(l)}$ is a learned weight matrix, and σ is a non-linearity. The degree normalisation matters for logistics networks because hubs like Cushing have many more connections than small inland feeder terminals, and without normalisation the hub's messages would dominate every aggregation.

C.3 Temporal extensions and hybrid architectures

Static graph architectures need extension for forecasting tasks where the time dimension matters. The relevant families are:

- **TGAT (Xu et al., 2020)** incorporates time encodings into the attention mechanism, weighting recent interactions more heavily than older ones.
- **TGN (Rossi et al., 2020)** maintains an updatable memory state for each node, capturing cumulative flow history — a vessel node's memory encodes its cargo history across voyages.
- **DCRNN (Li et al., 2018)** combines a spatial GNN with a temporal sequence model, treating spatial propagation as directed diffusion (physically appropriate for pipeline flows) and using a GRU encoder-decoder for multi-step forecasting.
- **STGCN (Yu et al., 2018)** interleaves graph convolution with dilated temporal convolution, achieving faster training while capturing both short-term fluctuations and seasonal patterns.

C.4 Why the framework is GNN-ready

The framework's resolved output is structurally what these architectures expect. The node feature matrix X(t) is built directly from scenario_resolved_values by pivoting on (variable_id, observation_date). The adjacency matrix A is the v_flow_edges view. The conservation constraint — mass balance at each node — can be enforced as an architectural prior, as a soft loss penalty during training, or as a hard projection layer after inference. The latent variables of Section 4.5 are the natural prediction targets: a model that learns to predict them from observed flows and historical patterns becomes the inference layer the framework's design anticipates.

The work required to put a GNN on top of the framework is well-defined: choose an architecture, define a train/validation/test split that does not leak across time, select a target variable, and select a baseline. These are the components of forecasting research, and they sit outside the scope of this thesis but on top of the substrate it provides.